

[.NET](#) / [REDIS](#) / [CACHE-ASIDE](#)

Redis caching in .NET

Bound the staleness.

A cache-aside read with an explicit expiration policy.

● ProductCache.cs / read path

C#

```
// cache: IDistributedCache; db: your data source
var key = $"product:{id}";
var json = await cache.GetStringAsync(key, ct);

if (json is null)
{
    var product = await db.FindAsync(id, ct);
    if (product is null) return Results.NotFound();

    json = JsonSerializer.Serialize(product);
    await cache.SetStringAsync(key, json, new()
    {
        AbsoluteExpirationRelativeToNow =
            TimeSpan.FromMinutes(5)
    }, ct);
}

return Results.Content(json, "application/json");
```

↳ DESIGN NOTE

A TTL limits staleness; it does not prevent it. Invalidate on writes when freshness matters, and coalesce concurrent misses to protect the database.

[.NET](#) / [REDIS](#) / [EXPIRATION](#)

Cache expiration

Before / After

A sliding window can keep a hot key alive indefinitely.

• BEFORE / sliding expiration only

C#



```
await cache.SetStringAsync(key, json, new()  
{  
    SlidingExpiration =  
        TimeSpan.FromMinutes(2)  
}, ct);
```

• AFTER / add an absolute lifetime

C#



```
await cache.SetStringAsync(key, json, new()  
{  
    SlidingExpiration =  
        TimeSpan.FromMinutes(2),  
    AbsoluteExpirationRelativeToNow =  
        TimeSpan.FromMinutes(10)  
}, ct);
```

↳ DESIGN NOTE

Sliding expiration removes idle entries. Absolute expiration caps each entry's lifetime. Invalidate on writes when freshness matters.

TYPESCRIPT / JAVASCRIPT

Same behaviour. Two languages.

Type annotations disappear at runtime.

● TypeScript

TYPESCRIPT

```
function greet(name: string): string {  
  return `Hello, ${name}`;  
}  
  
console.log(greet('Manjunath'));
```

● JavaScript

JAVASCRIPT

```
function greet(name) {  
  return `Hello, ${name}`;  
}  
  
console.log(greet('Manjunath'));
```

[.NET](#) / [PERFORMANCE](#) / [SPAN](#)

Slice, don't copy.

Span<T> in hot paths

The same parsing logic, with and without a new string per call.

Substring

CSHARP



```
static int ReadId(string line)
{
    var id = line.Substring(4, 6);
    return int.Parse(id);
}

static bool HasFlag(string line)
{
    var lower = line.ToLower();
    return lower.Contains("urgent");
}
```

- ✗ Allocates a new string per call
- ✗ Extra work for the garbage collector
- ✗ Copies bytes that already exist

Span

CSHARP



```
static int ReadId(ReadOnlySpan<char> line)
{
    var id = line.Slice(4, 6);
    return int.Parse(id);
}

static bool HasFlag(ReadOnlySpan<char> line)
{
    return line.Contains("urgent",
        StringComparison.OrdinalIgnoreCase);
}
```

- ✓ Works over the caller's memory
- ✓ No temporary strings
- ✓ Same readability

↳ DESIGN NOTE

Substring allocates a new string every call. Slicing a `ReadOnlySpan<char>` reuses the memory the caller already owns.

[.NET 9](#) / [THREADING](#) / [LOCKS](#)

Four ways to guard a resource

Concurrency in .NET

Pick by what you are protecting and whether the caller is async.

 lock statement

CSHARP

```
private readonly object _gate = new();

public void Increment()
{
    lock (_gate)
    {
        _count++;
    }
}
```

 System.Threading.Lock

CSHARP

```
private readonly Lock _gate = new();

public void Increment()
{
    lock (_gate)
    {
        _count++;
    }
}
```

 Semaphore

CSHARP

```
private static readonly Semaphore _slots = new(3, 3);

public void Access()
{
    _slots.WaitOne();
    try { DoWork(); }
    finally { _slots.Release(); }
}
```

 SemaphoreSlim

CSHARP

```
private static readonly SemaphoreSlim _slots = new(3);

public async Task AccessAsync()
{
    await _slots.WaitAsync();
    try { await DoWorkAsync(); }
    finally { _slots.Release(); }
}
```

↳ DESIGN NOTE

lock and Lock are for short critical sections on one process. Semaphores bound concurrency; only SemaphoreSlim can be awaited.

[.NET](#) / [DI](#) / [TESTABILITY](#)

Ask for it. Don't build it.

Constructor injection keeps OrderService testable and unaware of concrete mail delivery.

 Without DI

CSHARP



```
public class OrderService
{
    private readonly EmailService _email = new();
}
```

- Hard-wired to one implementation
- Cannot be unit tested in isolation

 With DI

CSHARP



```
public class OrderService(INotificationService notifications)
{
    private readonly INotificationService _notifications = notifications;
}
```

- Depends on an abstraction
- Swappable in tests and in production